



Sincronización Automática FAQ → Diagramas de Flujo

> Ficha técnica del mecanismo de 3 capas que mantiene `faq_higiene_estado_sesion_flujo.{tex,pdf}` alineados con `faq_higiene_estado_sesion.md`, sin intervención manual

1 Objetivo y motivación

El documento `docs/AGENTE_IA/faq_higiene_estado_sesion_flujo.tex` es una **traducción semántica** (no mecánica ni 1:1) del FAQ `faq_higiene_estado_sesion.md`. Los cinco diagramas ISO 5807 que contiene resumen la cadena de integridad, la guardia MCP por `mtime`, los veredictos de `estado_sesion.py` `check/clean` y el eslabón semántico de `bitacoras.py`.

El problema que motiva este flujo es de *consistencia*: si alguien edita el FAQ (añade un veredicto, cambia una fecha, corrige una frase), el diagrama queda obsoleto. Hacer la traducción a mano es lento, y la traducción “desde cero” por un LLM rompe la geometría de los diagramas (coordenadas, estilos y número de nodos son frágiles).

💡 Concepto clave

🕒 **Objetivo**: cuando se edite el `.md`, el flujo detecta el cambio por **hash SHA-256**, decide *qué* hay que traducir, lo traduce de forma **híbrida** (campos deterministas por script, semántica vía LLM quirúrgico), recompila el PDF y **verifica** el resultado sin necesidad de visión humana.

El resultado es un bucle de auto-curación determinista: la misma entrada (mismo `.md` en el mismo estado) produce siempre el mismo comportamiento, y la única parte probabilística (el LLM) se mantiene acotada y *validada* antes de tocar el documento.

2 Arquitectura de 3 capas

El proyecto ELECTRONICA impone una separación estricta de responsabilidades. Este flujo la respeta al pie de la letra:

A esta tríada se suma un cambio *aditivo* en la pieza compartida `execution/compile_latex.py`: un parámetro nuevo `clean=False` que permite al orquestador conservar el log compilación hasta después de la verificación (ver Sección 7).

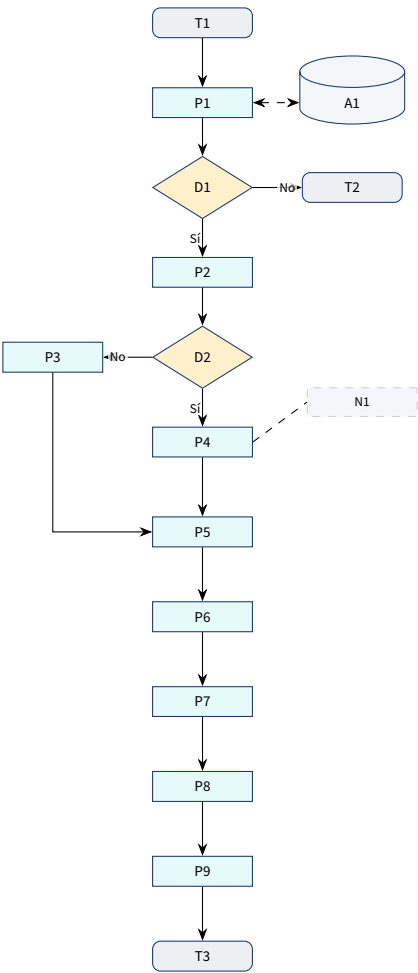
Capa	Artefacto	Rol (qué / cómo)
1. Directiva	directives/sync_faq_a_flujo.yaml	SOP de nivel medio: qué hacer, paso a paso, y qué hacer ante cada caso límite (edge case).
2. Orquestación	flujo_sync_faq_flujo.py	Toma de decisiones: comparar hash, planear, encadenar ejecución, avisar de coste, compilar, copiar, alertar.
3. Ejecución	execution/regenerar_faq_flujo.py execution/verificar_pdf.py	Cómo: scripts deterministas con CLI, salidas JSON y códigos de salida categorizados.

 **Reutilización antes que construcción.** El flujo no reimplementa nada: reutiliza `execution/enrutador.py` (decisión de tier), `execution/llm_client.py` (chat por OpenRouter), `execution/compile_latex.py` (compilación), `execution/alert_user.py` (notificación audible) y los patrones JSON ya documentados en `.agent/python.md`.

3 Visión general del flujo completo

3.1 La cadena, de principio a fin

Cada ejecución del orquestador recorre esta secuencia (fases E1–E5 del SOP):



Leyenda de etiquetas

Etiqueta	Significado
T1	Inicio: el orquestador arranca con <code>flujo_sync_faq_flujo.py -once</code> .
P1	Cálculo del hash SHA-256 del <code>faq_higiene_estado_sesion.md</code> (función pura determinista).
A1	Estado del flujo: <code>.tmp/faq_flujo_sync.json</code> (hashes md/tex, vía, secciones).
D1	¿El hash del <code>.md</code> coincide con el guardado? Sí → proceso inerte; No → clasificar.
T2	Fin sin cambios: el documento y el flujo quedan sincronizados.
P2	Clasificar el diff del <code>.md</code> contra el snapshot (<code>regenerar_faq_flujo.py -plan</code>).
D2	¿Cambio estructural (requiere LLM)? Sí → re-traducción; No → inyectar campos.
P3	Inyectar campos deterministas (regex): fecha, modelo obsoleto, número de bitácoras antiguas.
P4	Re-traducción quirúrgica vía LLM: tier decidido por <code>enrutador.py</code> , edits JSON validados.
N1	Aviso previo de consumo de créditos OpenRouter antes de invocar el LLM.
P5	Reescribir <code>faq_higiene_estado_sesion_flujo.tex</code> aplicando los edits validados.
P6	Compilar con 2 pasadas de <code>pdflatex</code> (log conservado vía <code>compile_latex.py clean=False</code>).
P7	Copiar el <code>.pdf</code> del build a <code>docs/AGENTE_IA/</code> .
P8	Guardrail post-compilación: 0 errores “!” en el log; overfull y solapes registrados.
P9	Estado + snapshot: actualiza <code>faq_flujo_sync.json</code> y el snapshot del <code>.md</code> ; alerta audible.
T3	Fin sincronizado: el flujo queda actualizado y compilado.

Nota: la decisión central “¿cambio estructural?” la toma `regenerar_faq_flujo.py` comparando el `.md` con su snapshot (Sección 5), no el chat del agente.

3.2 Geografía de archivos

Archivo	Papel
docs/AGENTE_IA/faq_higiene_estado_sesion.md	Fuente de verdad del FAQ (se edita).
docs/AGENTE_IA/faq_higiene_estado_sesion_flujo.tex	Plantilla de diagramas; se edita <i>in situ</i> ; nunca se genera desde cero.
docs/AGENTE_IA/faq_higiene_estado_sesion_flujo.pdf	Entregable compilado (entrada del usuario).
.tmp/faq_flujo_sync.json	Estado del flujo: hashes md/tex, marca temporal, vía, secciones, edits.
.tmp/faq_flujo_md_snapshot.md	Copia del .md en el último arranque (base del diff de clasificación).
.tmp/faq_flujo_sync.log	Log append-only (hora UTC) de cada corrida.
.tmp/latex_build/	Directorio de compilación (auxiliares + .pdf de build).

El estado en `.tmp/` es *derivado*: cualquiera de esos ficheros puede eliminarse y el flujo lo regenera en el siguiente arranque (la línea base se establece sola).

4 Detección de cambios por hash

El disparador no depende de inotify ni de watchers del RAG: es un **polling por hash** que también funciona con una sola ejecución puntual.

```
def _sha256(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()

estado = _leer_estado() # .tmp/faq_flujo_sync.json
md_sha = _sha256(md)
if not args.force and estado.get("md_sha256") == md_sha:
    return {..., "cambio": False, "message": "Sin cambios en el markdown (hash idéntico)."}

```

- `-force` salta la comparación y fuerza la cadena completa (equivalentemente, la primera ejecución sin estado previo también entra).
- `-watch [-interval N]` repite la comprobación cada *N* segundos (mínimo 5) en un bucle hasta Ctrl-C; cada corrida imprime su propio JSON y log.
- El hash también se guarda en el estado tras sincronizar, de modo que la siguiente corrida detecta *el próximo* cambio (no el mismo).

💡 Concepto clave

🔗 La detección por hash es **determinista** (función pura de los bytes del archivo) y **barata**: sin cambio no hay llm, ni compilación, ni créditos.

5 Clasificación del cambio: determinista vs. semántico

Antes de llamar a ningún LLM, el script `regenerar_faq_flujo.py` responde a la pregunta: *¿este cambio necesita traducción semántica o basta con una sustitución?*

5.1 Campos conocidos (inyección determinista)

Hay tres cosas del FAQ que se traducen por **regex exactas**, sin LLM:

Campo	Regex	Efecto en el .tex
Fecha de la plantilla	<code>FECHA_PLANTILLA (YYYY-MM-DD)</code>	Actualiza la fecha ISO y el encabezado día/mes/año (DD/MM/AAAA).
Modelo obsoleto	<code>deepseek-[\w.-]+</code>	Sustituye el identificador del modelo (deepseek-v4-pro → el vigente).
Bitácoras antiguas	<code>\d+\s+antiguas</code>	Ajusta el número “N antiguas” en la sección de bitácoras.

Estas dejan de tocar el LLM y devuelven `via: "determinista"`. El script aplica la sustitución en memoria y solo la escribe si hubo cambio real (`tex_cambiado`).

5.2 El diff contra el snapshot

Para lo demás se usa `difflib.SequenceMatcher` entre el `.md` actual y la copia `.tmp/faq_flujo_md_snapshot.md` del último arranque. Los *opcodes* de cambio se agrupan por sección del FAQ:

- Las cabeceras `## N. ...` se indexan con una regex y cada línea del diff se asigna a su sección (`_seccion_de_linea`).
- Una sección se marca `semantico=True` si alguna línea cambia después de *normalizar* los campos conocidos (`_scrub_line` reemplaza fecha/modelo/N por marcadores de posición), o si el número de líneas viejo $\neq$ nuevo.
- Las secciones “front” (metadatos) y “Referencias cruzadas” se marcan ignoradas.

```
def _scrub_line(line):
    line = re.sub(_RE_FECHA, "<FECHA>", line)
    line = re.sub(_RE_MODELO, "<MODELO>", line)
    line = re.sub(_RE_ANTIGUAS, "<N> antiguas", line)
    return line

for o, n in zip(old_lines, new_lines):
    if _scrub_line(o) != _scrub_line(n):
        entry["semantico"] = True          # cambio real de contenido
```

5.3 Mapeo sección del FAQ → bloque del .tex

El `.tex` se divide en 4 bloques usando marcadores de comentario propios (`%` seguido de dos líneas de caja, `U+2550`, y “Sección N:”), El bloque 4 termina en `\end{document}`:

```
_MARKERS = {n: "% \u2550\u2550 Secci\u00f3n " + str(n) + ":" for n in (1,2,3,4)}
def _blocks_tex(tex):
    pos = {n: tex.find(S) for n, S in _MARKERS.items()}
    end = tex.find("\end{document}")
    return {1: (pos[1], pos[2]), 2: (pos[2], pos[3]),
            3: (pos[3], pos[4]), 4: (pos[4], end)}
```

Así, un cambio en la sección 2 del FAQ solo dispara evaluación sobre el bloque 2 del `.tex`; el resto del documento queda intacto por construcción. Esta es una divergencia *consciente* entre la numeración de Markdown y la de bloques: los diagramas no siguen 1:1 los encabezados.

5.4 La salida del plan

Con `-plan` el script no llama al LLM ni escribe: devuelve el mapa de decisión. Ejemplo real tras un cambio en la sección 2:

```
{ "status": "ok", "tex_cambiado": false, "via": "plan",
  "known_fields": {},
  "cambios": [ {"seccion": "2", "via": "llm", "hunks": 1} ],
  "llm_necesario": true,
  "nota": "La sincronizacion real invocara el LLM via OpenRouter (consume creditos)
  ." }
```

El orquestador usa este veredicto para **avisar al usuario del coste antes** de consumir créditos, y para abortar limpiamente si se pasa `-no-llm` y el cambio es estructural.

6 Re-traducción semántica (LLM quirúrgico)

Cuando hay cambios estructurales, el flujo no regenera el bloque: lo *edita por parches*. El LLM recibe como plantilla el bloque `.tex` actual y produce un JSON de edits; el script valida y aplica. Así se preserva la geometría dibujada y ajustada manualmente.

6.1 Routing determinista (sin decisión en el chat)

El tier se decide con `execution/enrutador.py` a partir del descriptor `-task conversion` y los **tokens medidos** del corpus (markdown afectado + bloque `.tex` + inventario del diff):

```
task = "conversion"
decision = decide(task, tokens, critico, False, None) # funcion pura
candidates = [decision["model"]] + (decision.get("fallback") or [])[:2]
if model: # --modelo explicito
```

```
candidates = [model] + [c for c in candidates if c != model]
```

- Por defecto el tier es flash (google/gemini-3.7-flash, barato y suficiente para edición de tablas/texto).
- -critico escala a opus; el fallback cost-aware de la cadena (flash → deepseek → glm) solo se usa ante 429/errores, con retry budget máximo 3 intentos.
- Cada decisión se registra en .tmp/routing_log.jsonl (telemetría), y la llamada consume créditos de OPENROUTER_API_KEY vía execution/llm_client.openrouter_chat.

6.2 El prompt de editor quirúrgico

El system prompt fija reglas invariables; la más importante es *no tocar la geometría*. El user prompt entrega el cambio detectado (fragmentos ANTES/ AHORA) y el bloque actual entre marcadores.

```
system = (
    "Eres un editor quirurgico de codigo LaTeX ...\n"
    "1. NO modifiques la geometria: ni coordenadas, ni estilos \node[...], ni el\n"
    "   numero de "\n"
    "nodos, ni las etiquetas internas (T1, P1, D1, E1, A1, N1...).\n"
    "2. Solo edita textos: filas de la tabla 'Leyenda de etiquetas', parrafos\n"
    "   explicativos...\n"
    "3. Puedes anadir filas a la tabla (minimas) o reescribir significados. "\n"
    "PROHIBIDO anadir nodos o secciones.\n"
    "4. Manten el LaTeX valido: en tablas escapa & como \& y _ como \_ dentro de \n"
    "   texttt.\n"
    "5. Devuelve SOLO JSON: {\"edits\": [{\"old\": \"subcadena unica\", \"new\": \"texto\n"
    "   \"]}}. "\n"
    "Si no hay nada que cambiar, {\"edits\": []}.\n"
    "6. No escribas nada fuera del JSON."
)
```

6.3 De la respuesta cruda a edits seguros

1. `_parse_edits`: extrae el primer objeto JSON balanceado (`_find_balanced_json`, que respeta strings y escapes – ver `.agent/python.md`), elimina trailing commas y hace `json.loads`.
2. `_edits_validos`: exige que cada old sea una subcadena **única** del bloque y que el new no contenga tokens estructurales prohibidos (`\begin{tikzpicture}`, `\section`, `\documentclass`, ...). Cualquier incumplimiento aborta con error.
3. `_aplicar_edits`: sustitución estricta (re-valida unicidad antes de cada replace).
4. **Invariante geométrico**: tras aplicar todos los edits de un bloque, se cuenta `\node[` antes y después; si difieren, el script aborta *sin escribir* (código 5).

```
n_antes = block.count("\\node[")
n_despues = nuevo_block.count("\\node[")
if n_antes != n_despues:
```

```
raise RuntimeError("El LLM altero la geometria del bloque ... Abortando sin
escribir.")
```

6.4 Escritura atómica

Solo después de validar todos los bloques se escribe el fichero, y se hace de forma atómica (archivo temporal + `os.replace`) para no dejar nunca un `.tex` a medias:

```
if not (args.dry_run or not resultado["tex_cambiado"]):
    fd, tmp = tempfile.mkstemp(dir=str(tex_path.parent), suffix=".tmp")
    with os.fdopen(fd, "w", encoding="utf-8") as fh:
        fh.write(tex)
    os.replace(tmp, tex_path)
```

Un LLM que devuelva `{"edits": []}` (porque el bloque ya está al día) es un *resultado correcto*: la cadena valida el JSON vacío y continúa con `tex_cambiado: False`, sin recompilar.

7 Compilación y el parámetro `clean=False`

El orquestador compila el `.tex` regenerado con la pieza compartida `compile_latex_code` (2 pasadas de `pdflatex` para resolver referencias) al directorio `.tmp/latex_build/`:

```
res = compile_latex_code(contenido, job_name=JOB_NAME,
                        output_dir=str(BUILD_DIR), clean=False)
...
pdf_build = BUILD_DIR / (JOB_NAME + ".pdf")
shutil.copyfile(pdf_build, pdf)          # publica el entregable en docs/AGENTE_IA/
```

El parámetro nuevo `clean=False` es el único cambio tocado en la pieza compartida. ¿Por qué? Por defecto `compile_latex_code` borra los auxiliares (incluido el `.log`) al terminar – perfecto para el MCP de LaTeX, que solo necesita el PDF. Pero el guardrail de verificación necesita leer el `.log` (errores ! y Overfull). Con `clean=False` el orquestador conserva el log, ejecuta `verificar_pdf.py`, y **luego** limpia él mismo con `clean_latex_aux_files(BUILD_DIR)`.

🛡️ **Compatibilidad asegurada:** el cambio es aditivo con valor por defecto `True`. Ningún llamante existente (`mcp_latex_server.py` y los demás flujos) cambia de comportamiento.

8 Guardrail de verificación (sin visión humana)

`execution/verificar_pdf.py` es la validación determinista post-compilación. Tiene tres frentes:

1. **Errores de compilación:** líneas del `.log` que empiezan por `!`. Su presencia vuelve la salida `status:` `"error"` y código 1.
2. **Overfull \hbox:** se reportan (informativos, cosméticos) pero no bloquean.
3. **Solapes de texto:** parsea `pdftotext -bbox` (caja por palabra) y detecta pares de palabras cuyos rectángulos se solapan más de un umbral.

8.1 Cálculo del solape

```
SOLAPE_FRACCION = 0.35

def _solapa(r1, r2):
    dx = min(x2,u2) - max(x1,u1); dy = min(y2,v2) - max(y1,v1)
    if dx <= 0 or dy <= 0: return 0.0
    inter = dx * dy
    menor = min(area(r1), area(r2))
    return inter / menor          # fraccion del rectangulo MENOR que queda tapada
```

El umbral `0.35` significa que solo se reporta un solape cuando más del 35 % del rectángulo de la palabra menor queda cubierto por otra palabra – tolera ligeros contactos de caja (típicos en `TikZ` con `\ttk`) y captura colisiones reales (como la que se corrigió en el diagrama 1 durante el diseño). Con esto el orquestador puede *verificar sin visión*: reporta “6 páginas, 0 errores, 1 Overfull, 0 solapes” en el log y en el estado.

9 Estado, log y salida JSON

9.1 Códigos de salida

Código	Origen	Significado
0	orquestador	OK: sincronizado o sin cambios.
1	orquestador	Error de cadena (falta archivo, compilación fallida, LLM falló).
2	regenerar	Error de entrada (<code>.md/.tex/snapshot</code> inexistentes).
3	regenerar	Cambio estructural detectado con <code>-no-llm</code> : <code>.tex</code> intacto.
4	regenerar	<code>.tex</code> inexistente: el flujo <i>sincroniza</i> , no crea diagramas desde cero.
5	regenerar	Fallo LLM o edits inválidos tras el <code>retry budget</code> (nada escrito).

9.2 Ejemplos de salidas reales

Ejecución sin cambios:

```
{"status":"ok","cambio":false,"message":"Sin cambios en el markdown (hash idéntico)"}
.
```

Ejecución con cambio estructural atendido (del test E2E):

```
{
  "status": "ok",
  "cambio": true,
  "via": "llm",
  "tex_cambiado": true,
  "secciones": [
    {
      "seccion": "2",
      "via": "llm",
      "hunks": 1,
      "edits": 1
    }
  ],
  "edits": 1,
  "verificacion": {
    "status": "ok",
    "paginas": 6,
    "errores": [],
    "overfull": [
      "Overfull \\hbox (6.79pt too wide) ..."
    ],
    "solapes": [],
    "solapes_total": 0
  }
}
```

El estado persistido tras sincronizar (`.tmp/faq_flujo_sync.json`) contiene los hashes md/tex, la marca temporal UTC, la vía (determinista/llm/sin_cambios), las secciones atendidas, los campos conocidos inyectados y el número total de edits.

10 La directiva (Layer 1)

La fase 1 del SOP define las entradas requeridas y los 7 pasos; la fase de *edge cases* es la que hace robusto al flujo. Resumen de los casos límite documentados:

Caso límite	Protocolo
Sin snapshot previo (primer arranque)	Se toma línea base: campos deterministas + snapshot; NO se llama al LLM (no hay diff que interpretar).
Cambio solo de metadatos (tabla de commits, referencias cruzadas)	Se ignora; el hash queda marcado como atendido sin tocar el <code>.tex</code> .
Fallo LLM o edits inválidos	Retry por la cadena de fallback del router (máx 3); si se agota, código 5 sin escribir.
El LLM intenta alterar geometría	La validación “mismo número de <code>\node[</code> ” rechaza y aborta.
<code>-no-llm</code> con cambio estructural	Código 3; el <code>.tex</code> queda intacto y se reportan las secciones afectadas.
<code>.tex</code> inexistente	Código 4 (sincroniza, no crea desde cero).

11 Cómo se validó

- Sanidad estática:** `py_compile` de los tres scripts nuevos (se depuró un fallo de f-string con barra invertida en el conteo de `\node[`).
- Pruebas unitarias deterministas** sobre `_edits_validos` / `_aplicar_edits`: edit válido preserva el invariante de nodos; old no único es rechazado; token estructural prohibido es rechazado; y `aplicar_campos_conocidos` inyecta fecha/modelo/N correctos.
- Línea base real:** un primer arranque crea snapshot + estado; un segundo arranque con el `.md` intacto responde “hash idéntico” (sin LLM, sin compilación).
- E2E con LLM real** sobre copias en `/tmp` (sin tocar el entregable): un cambio semántico en la sección 2 fue clasificado como `llm`, el LLM devolvió 1 `edit` quirúrgico correcto (amplió la nota N1 con la resolución temporal de time, LaTeX válido), la compilación fue exitosa y el verificador reportó 6 páginas, 0 errores, 0 solapes. El estado real se restauró tras la prueba.

Concepto clave

- ✓ El E2E confirmó también el caso “*edits vacíos es correcto*”: el LLM devolvió `{"edits": []}` cuando el bloque ya estaba al día, lo que validó el parseo del JSON vacío y la continuación sin recompilar.

12 Uso

```
python3 flujo_sync_faq_flujo.py # sincronización puntual
python3 flujo_sync_faq_flujo.py --watch --interval 60 # bucle en segundo plano
python3 flujo_sync_faq_flujo.py --force # re-sincronizar ignorando el hash
python3 flujo_sync_faq_flujo.py --no-llm # solo parte determinista (aborta
    ante
                                # cambios estructurales, código
                                3)
python3 flujo_sync_faq_flujo.py --dry-run # muestra el plan sin escribir/
    compilar
python3 flujo_sync_faq_flujo.py --critico # re-traducción LLM en tier opus
python3 execution/regenerar_faq_flujo.py --plan # solo clasificar (sin LLM, sin
    escribir)
python3 execution/verificar_pdf.py --pdf build.pdf --log build.log # guardrail
    standalone
```

Coste: la parte determinista y la detección por hash no consumen créditos. Solo la re-traducción semántica consume OPENROUTER_API_KEY (el orquestador lo avisa antes de ejecutarla). Con `-api-backend` no hay opción de backend: regenerar usa `openrouter_chat` por diseño (tier decidido por el enrutador).

13 Limitaciones y trabajo pendiente

- El flujo **no crea diagramas desde cero** (código 4): la plantilla del diagrama es la base; el diseño de un diagrama nuevo sigue siendo tarea del orquestador con revisión visual del usuario.
- La verificación por bbox detecta solapes de *texto*, no de elementos gráficos puros (líneas, flechas); el invariante de nodos y la regla “solo edita textos” protegen la geometría, pero una revisión visual humana del PDF sigue siendo la garantía final.
- El Overfull cosmético (6.79 pt) pre-existente se reporta y se mantiene; no bloquea la cadena.
- *Mejora futura sugerida:* usar la telemetría de `.tmp/routing_log.jsonl` para afinar los umbrales de tokens de la re-traducción (p. ej. cuando una sección devuelve consistentemente `edits: []`).

🔗 En una línea: editas el FAQ → el orquestador detecta el cambio por hash → decide si es determinista o semántico → el LLM solo parchea lo necesario con validación estricta → recompila y verifica → el PDF queda publicado y el próximo cambio queda armado.

Documento generado el 12/09/2026. Ficheros fuente: flujo_sync_faq_flujo.py,
execution/regenerar_faq_flujo.py, execution/verificar_pdf.py, execution/compile_latex.py y
directives/sync_faq_a_flujo.yaml.